LLMOps: Problem Statements

Seven problems. Prices in the notebook are illustrative placeholders; always use the provider's current pricing page.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Choose the Approach

Level: Warm-up | **Concepts:** prompting vs RAG vs fine-tuning

Choose the approach for: answering from a changing policy library; always replying in a strict JSON format; classifying 2 million tickets a month; summarising in the company's tone; knowing today's exchange rate.

Sample run / expected result

```
Policy library: RAG
Strict JSON: structured outputs first, fine-tune if still inconsistent
2M tickets: fine-tuned small model
Exchange rate: a tool or API call
```

Hints

- Facts that change: retrieval or tools. Behaviour: prompting or fine-tuning.

Problem 2: Prepare Fine-tuning Data

Level: Easy | **Concepts:** chat JSONL, validation

Convert 60 labelled tickets into chat-format JSONL, validate every line (roles, non-empty content, allowed labels), remove PII and split 80/20.

Sample run / expected result

```
Valid: 60/60 | PII removed: 11 | train 48, validation 12
```

Hints

- Validate before uploading; one bad line can fail a job.

Problem 3: Prompt Registry

Level: Medium | **Concepts:** versioning, hashes, diffs

Store prompt versions with an id, a short hash and a change note. Show a diff between two versions and write a release manifest.

Sample run / expected result

```
answer_v7 (3f9a1c) -> answer_v8 (b72e01): '+ Answer in at most 3 sentences.'
```

Hints

- hashlib.sha256(text.encode()).hexdigest()[:6]

Problem 4: Cost per Feature

Level: Medium | **Concepts:** token accounting

From the simulated request log, compute total cost per feature and per model per day, and the top 5 most expensive requests.

Sample run / expected result

```
agent_research: 45% of cost from 9% of requests
```

Hints

- Look for features with long contexts or many calls per request.

Problem 5: Latency Percentiles

Level: Easy | **Concepts:** p50, p95

Report p50 and p95 latency per model and per feature. Which one breaks a 3 second p95 target?

Sample run / expected result

```
large-model p95: 7.0 s (breaks target)
small-model p95: 1.8 s
```

Hints

- df.groupby(...).latency_ms.quantile(0.95)

Problem 6: Caching

Level: Challenging | **Concepts:** exact cache, hit rate, savings

Simulate 5,000 questions where a few questions are very common. Add an exact cache and report hit rate and money saved.

Sample run / expected result

```
Hit rate 81% -> 974 model calls instead of 5,000
```

Hints

- Normalise questions (lowercase, strip spaces) before using them as cache keys.

Problem 7: Routing Trade-off

Level: Stretch | **Concepts:** cost vs quality

Route easy questions to the small model and hard ones to the large model. Compare cost and simulated quality for all-large, all-small and routed.

Sample run / expected result

```
all-large: cost 100%, quality 0.92
routed: cost 35%, quality 0.91 (simulated quality)
```

Hints

- Validate routing on your golden set before trusting it.

Kiroku (記録): keeping records